Package 'mlmr'

November 11, 2026

Title A Guided Interface for Mixed-Effects and Multilevel Models

Version 0.1.0

Description Provides a guided Shiny interface and reusable R helpers for specifying, fitting, explaining, and reporting mixed-effects and multilevel models with 'lme4' (Bates, Maechler, Bolker, and Walker, 2015, <doi:10.18637/jss.v067.i01>). The package supports grand-mean and cluster-mean centering (Enders and Tofighi, 2007, <doi:10.1037/1082-989X.12.2.121>), nested and crossed random-effects structures, interaction terms, level-by-level and combined equations, APA-style tables, diagnostics, reproducible code, and software citation support for manuscript workflows.

License MIT + file LICENSE

URL https://marcusharrisphd.com/mlmr/, https://github.com/MarcusHarrisUConn/mlmr

BugReports https://github.com/MarcusHarrisUConn/mlmr/issues

Encoding UTF-8

Language en-US

Depends R (>= 3.6.0)

Imports shiny,
bslib,
lme4,
ggplot2

Suggests haven,
knitr,
readxl,
rmarkdown,
testthat (>= 3.0.0)

Config/testthat/edition 3

VignetteBuilder knitr

Contents

mlmr-package 2
example_hsb 3
mlm_apa_tables 3
mlm_fit 5

mlm_formula 6
mlm_hlm8_parity 7
mlm_publication_bundle 8
mlm_software_table 9
mlm_spec 10
mlm_supported_models 12
run_mlmr 12

Index **14**

`mlmr-package` *A Guided Interface for Mixed-Effects and Multilevel Models*

Description

mlmr provides a guided Shiny interface and supporting R toolkit for fitting, understanding, and reporting mixed-effects and multilevel models with **lme4**.

Details

The package is designed for users who want an HLM-style graphical workflow while preserving transparent and reproducible R code. The app supports level-based predictor selection, centering decisions, fixed effects, random intercepts, random slopes, interactions, model diagnostics, APA-style tables, level-by-level equations, combined equations, Tau variance-covariance displays, and reproducible exports.

Launch the app with `run_mlmr()`.

Inspect the current production scope with `mlm_supported_models()`. Inspect HLM-style parity and validation evidence with `mlm_hlm8_parity()` and `mlm_validation_report()`.

Create manuscript and reproducibility materials with `mlm_publication_bundle()`.

To cite the package, use `citation("mlmr")`.

Author(s)

Marcus Harris

See Also

`run_mlmr`, `mlm_supported_models`, `mlm_hlm8_parity`, `mlm_validation_report`, `mlm_publication_bundle`

example_hsb *Generate an HSB-Style Multilevel Example Dataset*

Description

Generates the built-in HSB-style example dataset used by the **mlmr** Shiny app. The data include student-level predictors, school-level predictors, categorical predictors, grouping factors, and outcomes suitable for mixed-effects model demonstrations.

Usage

```
example_hsb(n_schools = 24, min_students = 18, max_students = 34,
  seed = 20260423)
```

Arguments

`n_schools`	Number of schools to simulate.
`min_students, max_students`	Minimum and maximum number of students per school.
`seed`	Random seed used to make the generated data reproducible. The caller's random-number state is restored afterwards, so calling `example_hsb()` does not change later random draws. Use `NULL` to draw from the current random-number stream instead.

Value

A data frame containing simulated student, school, district, and teacher variables.

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
str(dat)
```

mlm_apa_tables *Create Tables and Equations for a Fitted mlmr Model*

Description

Creates manuscript-oriented outputs from a fitted **mlmr** model. These helpers return APA-style tables, level-by-level equations, combined equations, and LaTeX equation displays.

Usage

```
mlm_apa_tables(result, format = c("list", "html", "latex"),
  exponentiate = FALSE)

mlm_equations(result)

mlm_latex_equations(result)
```

Arguments

result	A fitted result returned by mlm_fit.
format	Output format for APA tables: "list", "html", or "latex". The list format returns individual data frames; HTML and LaTeX formats return complete manuscript-oriented table documents.
exponentiate	Logical; if TRUE, the fixed-effects table for a generalized model with a logit or log link gains exponentiated coefficients and their intervals: odds ratios (OR) for logit models, incidence rate ratios (IRR) for Poisson and negative-binomial models, and exp(b) for other log-link models. Ignored for Gaussian models and other links.

Details

The fixed-effects table includes estimates, standard errors, large-sample test statistics, p values, and Wald 95 percent confidence intervals. For Gaussian mixed models fit with **lme4**, p values are large-sample normal approximations rather than denominator-degrees-of-freedom tests. For generalized mixed models, p values and intervals are Wald approximations on the link scale. Treat these values as screening evidence unless a confirmatory inferential method is selected and documented.

Classical ICC tables are returned for Gaussian mixed models. For generalized mixed models, **mlmr** suppresses classical ICC output and asks users to choose a model-family-specific latent- or observation-scale ICC method before reporting ICC values.

mlm_latex_equations() writes two- and three-level nested models in hierarchical (level-by-level) notation. Predictors are assigned to levels from the fitted data: a predictor that is constant within every unit of a grouping factor is treated as measured at that level. Level-1 predictors appear in the Level-1 equation, higher-level predictors appear in the intercept equation, and cross-level interactions appear in the equation for the lower-level slope they moderate, so γ_{11} is the effect of the first Level-2 predictor on the first Level-1 slope. Equations are built from the fitted coefficients, so each dummy code of a categorical predictor has its own coefficient and the indices of the random effects in the Tau matrices match the equations. Three-level models use π, β, and γ coefficients. Generalized models are written for the linear predictor η with the link function shown and no Level-1 residual. Models with crossed grouping factors, or more than three levels, are written as a single mixed-model equation instead. These outputs are meant to support manuscript preparation and teaching; users should review notation and substantive labels before publication.

Value

mlm_apa_tables() returns a named list of tables, an HTML document, or a LaTeX document. mlm_equations() returns plain-text equation descriptions. mlm_latex_equations() returns a list with the level labels (level), one LaTeX string per level (equations), the combined equation (combined), one Tau matrix display per grouping factor (tau), a coefficient key linking each equation symbol to a fixed-effect estimate (key), the notation used (notation), and a short explanatory note (note).

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
spec <- mlm_spec(
  outcome = "mathscore",
  fixed = list(ses = list(center = "CWC")),
  grouping = list(schoolid = "schoolid"),
  random = list(schoolid = list(intercept = TRUE, slopes = character(),
    correlation = TRUE)),
```

```
  data = dat
)
result <- mlm_fit(spec)
mlm_apa_tables(result)
mlm_latex_equations(result)
```

`mlm_fit` *Fit a Multilevel Model Specification*

Description

Centers requested predictors, builds an **lme4** formula, and fits the specified mixed-effects or multilevel model.

Usage

```
mlm_fit(spec, REML = TRUE, optimizer = "bobyqa", maxfun = 20000)

## S3 method for class 'mlm_result'
print(x, ...)
```

Arguments

`spec`	An `"mlm_spec"` object created with `mlm_spec`.
`REML`	Logical; whether to use restricted maximum likelihood for Gaussian models. Use `REML = FALSE` when comparing nested models that differ in fixed effects.
`optimizer`	Optimizer passed to **lme4**, such as `"bobyqa"`, `"Nelder_Mead"`, or `"nloptwrap"`.
`maxfun`	Maximum number of function evaluations.
`x`	An `"mlm_result"` object returned by `mlm_fit()`.
`...`	Ignored.

Details

For Gaussian models, `mlm_fit()` calls `lme4::lmer()`. For supported generalized mixed models, it calls `lme4::glmer()` or `lme4::glmer.nb()`. The returned `fit` element is the underlying **lme4** model object, so users can pass it to standard **lme4** methods.

The function performs requested grand-mean and cluster-mean centering before fitting. Centering uses the analysis sample: means are computed from the rows that are complete on every model variable, which are the rows **lme4** keeps. A grand-mean-centered predictor therefore has a mean of zero, and a cluster-mean-centered predictor has cluster means of zero, among the observations that enter the model. Rows with missing values stay in the returned `data` but are not used for estimation.

`mlm_fit()` stops with an explanatory message when a specification cannot be fit as requested, for example when a categorical predictor is centered, a random slope is requested for a variable that is not a fixed predictor, or no random effects are specified. Users should inspect convergence messages, singular fits, residual patterns, and whether the random-effects structure is supported by the data.

Value

An object of class "mlm_result": a list with the fitted **lme4** model (fit), the model formula (formula), the data including centered predictors (data), R code that recreates the centered predictors (centering_code), the model specification (spec), and the estimation settings (settings: REML, optimizer, and maxfun). Printing the object shows a compact summary with the fixed-effects table.

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
spec <- mlm_spec(
  outcome = "mathscore",
  fixed = list(ses = list(center = "CWC")),
  grouping = list(schoolid = "schoolid"),
  random = list(schoolid = list(intercept = TRUE, slopes = character(),
    correlation = TRUE)),
  data = dat
)
result <- mlm_fit(spec)
result
summary(result$fit)
```

---

mlm_formula *Build an lme4 Formula From an mlmr Specification*

Description

Converts an **mlmr** model specification into an **lme4**-compatible model formula, including centered predictor names, interactions, and random effects.

Usage

```
mlm_formula(spec)
```

Arguments

spec	An "mlm_spec" object created with mlm_spec.

Details

The formula uses the effective predictor names created by the centering choices. For example, a predictor with center = "CWC" appears with a _CWC suffix. Correlated random slopes are represented with (1 + slope | group) syntax, whereas independent random slopes are represented with separate (1 | group) and (0 + slope | group) terms.

Value

A model formula.

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
spec <- mlm_spec(
  outcome = "mathscore",
  fixed = list(ses = list(center = "CWC")),
  grouping = list(schoolid = "schoolid"),
  random = list(schoolid = list(intercept = TRUE, slopes = "ses",
    correlation = TRUE)),
  data = dat
)
mlm_formula(spec)
```

`mlm_hlm8_parity` *Inspect HLM-Style Feature Parity and Validation Benchmarks*

Description

Summarizes **mlmr**'s progress toward an HLM-style graphical workflow and provides lightweight benchmark model specifications for validation.

Usage

```
mlm_hlm8_parity()

mlm_validation_benchmarks(run = FALSE, seed = 20260423, tolerance = 0.5)

mlm_validation_report(seed = 20260423, tolerance = 0.5)
```

Arguments

`run`	Logical; if `TRUE`, fit each benchmark model and report AIC values. The default `FALSE` only reports formulas and readiness checks.
`seed`	Numeric seed used to generate the deterministic example data.
`tolerance`	Maximum acceptable absolute AIC difference when `run = TRUE`.

Details

`mlm_hlm8_parity()` returns a table mapping common HLM 8 workflows to the current **mlmr** support level, evidence, and next development step.

`mlm_validation_benchmarks()` builds deterministic model specifications for core teaching and validation workflows: null, random-intercept, random-slope, cross-level interaction, and three-level models. With `run = TRUE`, the helper fits each model with `mlm_fit()` and compares the resulting AIC with stored deterministic reference values.

`mlm_validation_report()` fits the benchmark suite and returns a named list containing the validation summary, fixed effects, variance components, ICCs, and diagnostics. This is intended as reviewer-facing evidence for CRAN, software-paper, and classroom validation workflows.

Value

`mlm_hlm8_parity()` and `mlm_validation_benchmarks()` return data frames. `mlm_validation_report()` returns a named list of data frames and metadata.

Examples

```
mlm_hlm8_parity()
mlm_validation_benchmarks()
mlm_validation_benchmarks(run = TRUE)
names(mlm_validation_report())
```

`mlm_publication_bundle` *Create Publication and Reproducibility Materials*

Description

Creates publication-oriented reproducibility materials for a fitted **mlmr** model.

Usage

```
mlm_reproducibility_check(result, REML = NULL, optimizer = NULL,
  maxfun = NULL)

mlm_methods_text(result, REML = NULL, optimizer = NULL, maxfun = NULL)

mlm_publication_bundle(result, REML = NULL, optimizer = NULL,
  maxfun = NULL)
```

Arguments

`result`	A fitted result returned by `mlm_fit`.
`REML`, `optimizer`, `maxfun`	Estimation settings to report. These are normally left as `NULL`: the estimator is read from the fitted model and the optimizer settings from the values recorded by `mlm_fit`, so the reported settings cannot contradict the model. The arguments are used only for results that do not carry recorded settings.

Details

`mlm_reproducibility_check()` returns a checklist covering the formula, data footprint, estimator settings, centering decisions, random-effects structure, convergence, singularity, reporting outputs, and software versions.

The reproducible R code in the bundle is a complete script: given the original data frame as `dat`, it selects the analysis sample, recreates the centered predictors, and refits the same model.

`mlm_methods_text()` returns copy-ready prose that should be reviewed and adapted before publication. `mlm_publication_bundle()` combines the methods text, reproducibility checklist, software statement, formula, APA tables, equations, and reproducible R code into a named list.

Value

`mlm_reproducibility_check()` returns a data frame. `mlm_methods_text()` returns a character string. `mlm_publication_bundle()` returns a named list of publication materials.

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
spec <- mlm_spec(
  outcome = "mathscore",
  fixed = list(ses = list(center = "CWC")),
  grouping = list(schoolid = "schoolid"),
  random = list(schoolid = list(intercept = TRUE, slopes = character(),
    correlation = TRUE)),
  data = dat
)
result <- mlm_fit(spec)
mlm_reproducibility_check(result)
mlm_methods_text(result)
names(mlm_publication_bundle(result))
```

---

mlm_software_table *Report R and Package Versions*

Description

Creates a reproducibility-oriented table and APA-style prose statement reporting the R version and package versions used in an analysis.

Usage

```
mlm_software_table(packages = mlm_default_packages())

mlm_software_apa(packages = mlm_default_packages())

mlm_papaja_code(packages = mlm_default_packages(), bib_file = "r-references.bib")
```

Arguments

packages	Character vector of software or R package names to report. Use "R" to include the R version.
bib_file	Name of the BibTeX file to create or update when using the optional **papaja** workflow.

Details

mlm_papaja_code() returns R code that users can run when **papaja** is installed. The generated code uses papaja::r_refs() to create package references and papaja::cite_r() to create an APA-style R/package citation statement. **papaja** is optional and is not required to install or run **mlmr**.

Value

mlm_software_table() returns a data frame. mlm_software_apa() returns a manuscript-ready prose statement. mlm_papaja_code() returns character lines of reproducible R code.

Examples

```
mlm_software_table()
mlm_software_apa()
cat(mlm_papaja_code(), sep = "\n")
```

`mlm_spec` *Create a Multilevel Model Specification*

Description

Creates a structured model specification used by `mlm_formula`, `mlm_fit`, the Shiny app, and reporting helpers.

Usage

```
mlm_spec(outcome, distribution = "gaussian", link = "identity",
  fixed = list(), grouping = list(), nesting = NULL, structure = "nested",
  random = list(), interactions = list(), predictor_levels = list(),
  data = NULL)

## S3 method for class 'mlm_spec'
print(x, ...)
```

Arguments

`outcome`	Character string giving the name of the outcome variable in `data`.
`distribution`	Outcome distribution. The primary production workflow uses `"gaussian"`. Advanced workflows may use `"binomial"`, `"poisson"`, `"negative binomial"`, or `"Gamma"`.
`link`	Link function for generalized mixed models. For Gaussian models the default `"identity"` link is used.
`fixed`	Named list of fixed predictors. Each entry may include a `center` value: `"none"`, `"GMC"` for grand-mean centering, or `"CWC"` for cluster-mean centering. Only numeric predictors can be centered. Cluster-mean centering uses the first grouping factor unless the entry names another one with `group`, for example `list(center = "CWC", group = "districtid")`.
`grouping`	Named list mapping conceptual grouping names to data columns. For example, `list(schoolid = "schoolid")` or `list(schoolid = "schoolid", districtid = "districtid")`.
`nesting`	Optional text description of the nesting or design structure.
`structure`	Model structure label, such as `"nested"` or `"crossed"`. This label is used for documentation and guidance; the fitted model is determined by the grouping and random-effect terms.
`random`	Named list describing random intercepts, slopes, and correlation structure for each grouping factor. Each entry may contain `intercept = TRUE`, `slopes = c(...)`, and `correlation = TRUE` or `FALSE`.
`interactions`	List of character vectors describing interaction terms, for example `list(c("ses", "sector"))`.

`predictor_levels`
: Optional named list declaring conceptual predictor levels, such as `list(level1 = c("ses"), level2 = c("meanses"))`. This is used for app display and reporting.

`data`
: Data frame used to fit the model.

`x`
: An `"mlm_spec"` object.

`...`
: Ignored.

Details

`mlm_spec()` records the modeling choices made in the app or in code. It does not fit the model by itself. Centered variables are created during `mlm_fit` and are reflected in the generated **lme4** formula.

Variable names that are not syntactic R names (for example names containing spaces) are quoted automatically in the generated formula.

`mlm_spec()` checks that the selected variables exist in `data`. Problems that only matter at fitting time, such as centering a categorical predictor, requesting a random slope for a variable that is not a fixed predictor, or specifying no random effects, are reported by `mlm_fit` with a message describing how to fix them.

Users are responsible for confirming that grouping IDs, predictor levels, centering decisions, interactions, and random effects match the research design.

Value

An object of class `"mlm_spec"`, represented as a structured list. It can be passed to `mlm_formula` or `mlm_fit`.

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
spec <- mlm_spec(
  outcome = "mathscore",
  fixed = list(
    ses = list(center = "CWC"),
    meanses = list(center = "GMC"),
    sector = list(center = "none")
  ),
  grouping = list(schoolid = "schoolid"),
  random = list(schoolid = list(intercept = TRUE, slopes = "ses",
    correlation = TRUE)),
  interactions = list(c("ses", "sector")),
  predictor_levels = list(level1 = "ses", level2 = c("meanses", "sector")),
  data = dat
)
mlm_formula(spec)
```

mlm_supported_models *Summarize the Current mlmr Modeling Scope*

Description

Returns a table describing the modeling, reporting, import, diagnostic, and export features that are currently supported, experimental, or planned in **mlmr**.

Usage

```
mlm_supported_models()
```

Details

This helper is intended for transparent documentation and teaching. It gives users a compact way to see the current production path, advanced workflows, and areas that require extra statistical judgment or future development.

Value

A data frame with columns `Area`, `Status`, `Scope`, and `User_responsibility`.

Examples

```
mlm_supported_models()
```

run_mlmr *Launch the mlmr Shiny App*

Description

Launches the **mlmr** Shiny interface for mixed-effects and multilevel modeling with **lme4**.

Usage

```
run_mlmr(launch.browser = TRUE, port = getOption("shiny.port", NULL))
```

Arguments

`launch.browser`	Logical; whether to open the app in the default browser.
`port`	Optional port passed to `shiny::runApp`.

Details

The app opens with a built-in HSB-style example dataset and a preset model so users can test the complete workflow before uploading their own data. Uploaded data can be provided as CSV, TSV/TXT, Excel, SPSS, SAS, or Stata files when the optional import packages are installed.

The app is intended to guide model specification, estimation, diagnostics, equation display, APA-style tables, and reproducible exports. The strongest current production path is Gaussian two-level and three-level nested mixed-effects models. Use `mlm_supported_models()` to inspect the current supported, experimental, and planned scope.

Value

Runs the Shiny application. This function is called for its side effect and does not return a modeling object.

See Also

`mlm_supported_models`, `mlm_spec`, `mlm_fit`

Examples

```
if (interactive()) {
  run_mlmr()
}
```

Index

∗ **package**
mlmr-package, 2

example_hsb, 3

mlm_apa_tables, 3
mlm_equations *(*mlm_apa_tables*)*, 3
mlm_fit, *4*, 5, *7, 8*, *10, 11*, *13*
mlm_formula, 6, *10, 11*
mlm_hlm8_parity, *2*, 7
mlm_latex_equations *(*mlm_apa_tables*)*, 3
mlm_methods_text
*(*mlm_publication_bundle*)*, 8
mlm_papaja_code *(*mlm_software_table*)*, 9
mlm_publication_bundle, *2*, 8
mlm_reproducibility_check
*(*mlm_publication_bundle*)*, 8
mlm_software_apa *(*mlm_software_table*)*, 9
mlm_software_table, 9
mlm_spec, *5, 6*, 10, *13*
mlm_supported_models, *2*, 12, *12, 13*
mlm_validation_benchmarks
*(*mlm_hlm8_parity*)*, 7
mlm_validation_report, *2*
mlm_validation_report
*(*mlm_hlm8_parity*)*, 7
mlmr *(*mlmr-package*)*, 2
mlmr-package, 2

print.mlm_result *(*mlm_fit*)*, 5
print.mlm_spec *(*mlm_spec*)*, 10

run_mlmr, *2*, 12